Actively Learning Joint Contours of Multiple Computer Experiments

Shih-Ni Prim* Kevin R. Quinlan† Paul Hawkins† Jagadeesh Movva†
Annie S. Booth‡

August 24, 2026

arXiv:2512.13530v2 [stat.ME] 21 Aug 2026

Abstract

Contour location—the process of sequentially training a surrogate model to identify the design inputs that result in a pre-specified response value from a single computer experiment—is a well-studied active learning problem. Here, we tackle a related but distinct problem: identifying the input configuration that returns pre-specified values of multiple computer experiments simultaneously. Motivated by computer experiments of the rotational torques acting upon a vehicle in flight, we aim to identify stable flight conditions that result in zero torque forces. We propose a "joint contour location" (jCL) scheme that strikes a strategic balance between exploring the multiple response surfaces while exploiting learning of the intersecting contours. Rather than working exploration and exploitation into a single acquisition function, we devise two distinct acquisition schemes with a decision rule to choose between the two, which also provides a natural stopping criterion if no solution is present. We employ traditional Gaussian processes (GPs), multitask GPs, and deep GPs, but our jCL procedure is applicable to any surrogate that can provide posterior predictive distributions. Our jCL designs significantly outperform existing (single response) CL strategies, optimization-based alternatives, and previous strategies for targeting joint contours, enabling us to efficiently locate the optimal configurations for our motivating computer experiments.

Keywords: aerodynamics, contour location, Gaussian process, deep Gaussian process, surrogate, sequential design

1 Introduction

Computer simulation experiments are invaluable tools in many scientific fields, but particularly in the study of flight dynamics where physical experimentation is impractical across the large design space. (e.g., Pamadi et al., 2004; Vassberg et al., 2008; Economon et al., 2016; Quinlan et al., 2024). Here, our focus is on identifying a joint contour of multiple computer experiments. Specifically, let $f^{(r)} : \mathcal{X} \to \mathbb{R}$ denote the rth computer experiment for $r = 1, \ldots, R$. Each $f^{(r)}$ is deterministic, continuous, and acts on the same domain $\mathcal{X} \subset \mathbb{R}^d$. Our goal is to identify the "optimal design point":

$$\left\{ \tilde{\mathbf{x}} \in \mathcal{X} \;\middle|\; f^{(r)}(\tilde{\mathbf{x}}) = \tau_r \;\forall\; r = 1, \cdots, R \right\}, \tag{1}$$

where τ_r are pre-specified target response values. Throughout, we will focus on identifying a single $\tilde{\mathbf{x}} \in \mathcal{X}$, although we will discuss cases where zero or multiple solutions exist.

For example, we are motivated by high-fidelity computational fluid dynamics (CFD) simulations of a "High-Speed Army Reference Vehicle" (HARV) in flight (Vasile et al., 2022). Multiple CFD simulations

*Department of Statistics, NC State University
†Lawrence Livermore National Laboratory
‡Corresponding author: Department of Statistics, Virginia Tech, annie_booth@vt.edu

return the rotational torques acting on the vehicle (i.e., roll, pitch, and yaw moments) as functions of the orientation of the vehicle's fins (visualized later in Figure 8) and several flight condition parameters (Mach speed, angle of attack, angle of sideslip, and deflections of control surfaces). Stable flight occurs when all rotational torques acting on the vehicle are zero, also known as the "trim condition." Ultimately, we aim to compile an accurate trimmed aerodynamic database containing fin orientations that result in stable flight for various flight conditions. In this work we consider fixed positions of the top and bottom fins, and focus on identifying the optimal configuration of the side fins that results in zero pitch and zero roll. For a given sideslip angle, there will be only one such possible configuration. Efficient identification of the optimal fin configuration will facilitate application across a wide variety of settings. Given the dimensionality of this motivating application with relatively few movable fins, we restrict our exercises to low dimensional settings and discuss extensions to larger dimensions in Section 6.

Identifying $\tilde{\mathbf{x}}$ is particularly challenging when simulations are computationally expensive and evaluation budgets are limited. We seek a sample efficient method for identifying $\tilde{\mathbf{x}}$ from as few evaluations of the expensive $f^{(r)}$ as possible. This task requires a surrogate: a statistical model trained to emulate a computer experiment from limited training data. Effective surrogates provide accurate predictions at unobserved inputs with appropriate uncertainty quantification (UQ). Gaussian process (GP) surrogates are favored as they offer nonlinear regression with closed-form posterior predictive distributions in a Bayesian framework (Santner et al., 2003; Rasmussen and Williams, 2006; Gramacy, 2020). Multitask GPs can incorporate correlations among multiple responses (Bonilla et al., 2007), which can improve performance if the $f^{(r)}$ are not independent. Recent advances have aimed to retain the desirable properties of GPs while additionally incorporating nonstationary flexibility (Booth et al., 2024), with deep Gaussian processes (DGPs; Damianou and Lawrence, 2013) jumping to the forefront (e.g., Rajaram et al., 2020; Marmin and Filippone, 2022; Sauer, 2023; Ming et al., 2023; Yazdi et al., 2024), but multitask DGP surrogates have yet to be explored. Crucially, our active learning procedure is agnostic to the choice of surrogate (as long as posterior predictive distributions are available) and can leverage any of these GP variants.

When evaluations are limited, training data may be strategically collected through an iterative model-informed process called active learning (AL). In contrast to static one-shot experimental designs like Latin hypercube samples (LHS; McKay et al., 2000), active learning alternates between selecting new design points based on an existing surrogate and retraining the surrogate with the newly collected data. New design points are chosen by optimizing an acquisition function which objectively quantifies the utility of potential design points based on existing surrogate knowledge. Acquisition criteria may be tailored to particular objectives such as variance reduction (e.g., Cohn et al., 1996; Binois et al., 2019; Song and Joseph, 2026), optimization (e.g., Jones et al., 1998; Pourmohamad and Lee, 2021), calibration (e.g., Koermer et al., 2024; Sürer, 2025), or sensitivity analysis (e.g., Wycoff et al., 2021; Belakaria et al., 2024).

Our motivating problem is most akin to that of contour location (CL). Contour location is a type of active learning where the acquisition function specifically targets a level set, $\{\mathbf{x} \in \mathcal{X} \mid f(\mathbf{x}) = \tau\}$, and is designed to balance exploitation of the contour with exploration of the design space. Previous CL works have adapted common Bayesian optimization procedures (e.g., Ranjan et al., 2008; Picheny et al., 2010) or stepwise uncertainty reduction strategies (e.g. Bect et al., 2012; Chevalier et al., 2014; Marques et al., 2018) to target level sets instead of optima. More recent works have employed variations of classification entropy as acquisition criteria (e.g., Cole et al., 2023; Booth et al., 2025). While there is a vast body of literature on CL for single computer experiments, methods targeting joint contours of multiple experiments are scarce. Graziani and Ngom (2024) propose an adaptive design strategy for identifying inputs of multiple stochastic computer simulations whose response values fall within specified tolerances with sufficient probability. Their acquisition function is built upon Gaussian probability density functions (PDFs) centered at the target value for each response, but much of their machinery is inspired by the high

noise level of each simulation they consider. They target ranges $[\tau_r - \epsilon_r, \tau_r + \epsilon_r]$ instead of precise τ_r, and their surrogates can benefit from replicate observations to inform noise estimation. In our deterministic setting, we benefit from more aggressive targeting of τ_r and must intentionally discourage replicates. We find the PDF-based acquisition scheme to be too conservative for our settings (more on this in Section 4).

In order to identify $\tilde{\mathbf{x}}$ in Eq. (1) from as few evaluations of $f^{(r)}(\mathbf{x})$ as possible, we propose a novel acquisition scheme which we term "joint contour location" (jCL). Our acquisition procedure involves two mechanisms: one that aggressively exploits identification of the optimal design point and one that promotes exploration of all response surfaces, with a decision rule to decide which mechanism to trigger at each acquisition. This contrasts previous works that aim to merge exploration and exploitation into a single acquisition function. When surrogate certainty in the localization of $\tilde{\mathbf{x}}$ is low (which is common at the start of the design phase), we promote exploration of all response surfaces by targeting the Pareto front of posterior predictive uncertainty across all $r = 1, \ldots, R$. When surrogate certainty in the localization of $\tilde{\mathbf{x}}$ is high, we prioritize exploitation using the joint posterior probability of each $f^{(r)}(\mathbf{x})$ being within a yet unattained distance of τ_r. Our exploitation strategy is designed to quickly find a single $\tilde{\mathbf{x}}$ even if multiple exist; we defer extensions to ensure exploration of multiple equivalent solutions for future work. Our "decision tree" framework provides a natural stopping criterion if no solution exists: simply halt the design if too many exploration steps have been triggered in a row without any progress towards exploitation. Crucially, our procedure is agnostic to the choice of surrogate as long as posterior predictive probabilities are available. We will benchmark our methodology against a variety of competing strategies using several GP-based surrogates to demonstrate its applicability.

The remainder of this manuscript is organized as follows. Section 2 reviews surrogate modeling essentials while setting up an illustrative example. Section 3 details our jCL procedure. We validate our methodology on several synthetic exercises in Section 4 before deploying it on our motivating problem in Section 5. Section 6 concludes with discussion of relevant extensions of this work.

2 Gaussian Process Fundamentals

Let $\mathbf{x}_i \in \mathcal{X} \subset \mathbb{R}^d$ denote a d-dimensional design point. For a single black-box function, denote scalar output as $y_i = f(\mathbf{x}_i)$. Let $\mathbf{X}_n$ denote the $n \times d$ matrix of n row-combined design points, and let $\mathbf{y}_n$ denote the corresponding response vector.

A Gaussian process is a potentially infinite collection of random variables, any finite subset of which is distributed as a multivariate Gaussian distribution (Rasmussen and Williams, 2006). A standard Gaussian process prior on $f(\mathbf{x})$ assumes $\mathbf{y}_n \sim \mathcal{N}_n(\boldsymbol{\mu}_n, \Sigma(\mathbf{X}_n))$ for any design points $\mathbf{X}_n$. Without loss of generality, we assume $\boldsymbol{\mu}_n = 0$ after centering responses. The covariance matrix $\Sigma(\mathbf{X}_n)$ contains elements $\Sigma(\mathbf{X}_n)^{(ij)} = k(\mathbf{x}_i, \mathbf{x}_j)$ for $i \in \{1, \ldots, n\}$, $j \in \{1, \ldots, n\}$ where kernel k determines the covariance between y_i and y_j based on $\mathbf{x}_i$ and $\mathbf{x}_j$. Standard kernels, like the squared exponential and Matèrn (Stein, 1999), are inverse functions of Euclidean distance, encoding the assumption that inputs closer to each other are more likely to have similar outputs. Throughout, we use $\Sigma(\mathbf{A}, \mathbf{B})$ to represent the matrix with ij^{th} element containing the covariance between the ith row of $\mathbf{A}$ and the jth row of $\mathbf{B}$. We also use $\Sigma(\mathbf{A})$ as shorthand for $\Sigma(\mathbf{A}, \mathbf{A})$.

Conditioned on observed $\{\mathbf{X}_n, \mathbf{y}_n\}$, the posterior predictive distribution at a new location $\mathbf{x}^\star$ follows

$$f_n(\mathbf{x}^\star) \mid \mathbf{X}_n, \mathbf{y}_n \sim \mathcal{N}_1(\mu, \sigma^2) \quad \text{where} \quad \begin{cases} \mu = \Sigma(\mathbf{x}^\star, \mathbf{X}_n)\Sigma(\mathbf{X}_n)^{-1}\mathbf{y}_n \\ \sigma^2 = \Sigma(\mathbf{x}^*) - \Sigma(\mathbf{x}^*, \mathbf{X}_n)\Sigma(\mathbf{X}_n)^{-1}\Sigma(\mathbf{X}_n, \mathbf{x}^*). \end{cases} \tag{2}$$

Posterior probabilities over intervals may be obtained through the application of the standard Gaussian cumulative distribution function (CDF).

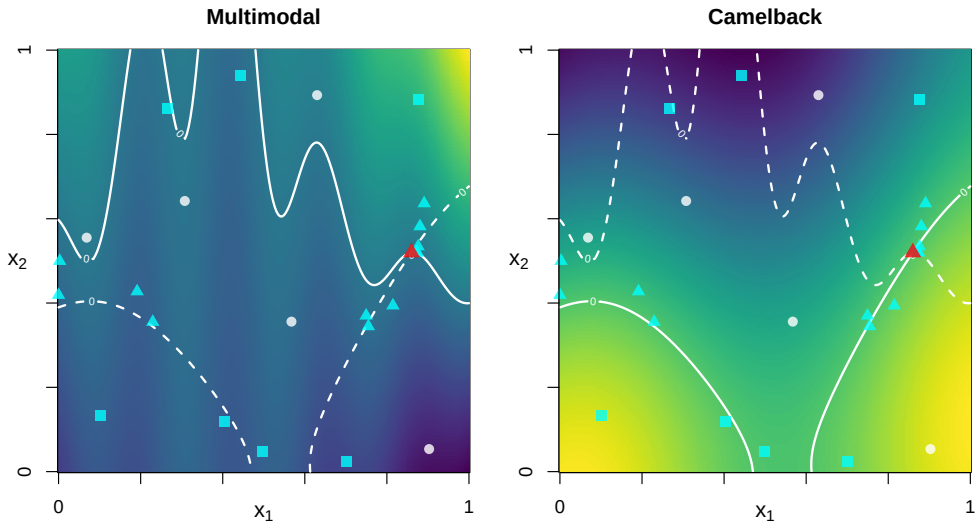


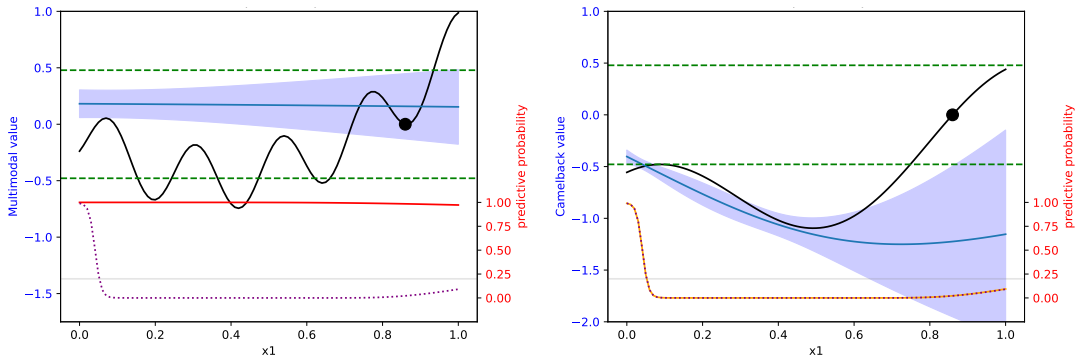
Figure 1: Heatmaps (yellow/high, purple/low) of the 2d multimodal function (left; Bichon et al., 2008) and the 2d camelback function (right; derived from Molga and Smutnicki, 2005). Solid white lines show the respective contours $f^{(r)}(\mathbf{x}) = 0$. Red triangle marks the optimal design point where both contours are zero. White circles indicate an initial LHS, and cyan squares (exploration) and triangles (exploitation) indicate acquisitions made by our jCL scheme.

To demonstrate, consider the two-dimensional multimodal and camelback functions shown in Figure 1 with $\tau_1 = \tau_2 = 0$ (after scaling).[1] We observed both functions at the same 5 input locations (the random LHS indicated by the white circles), then trained independent GPs for each surface. The upper panels of Figure 2 show the GP posterior means (solid blue) and 95% credible intervals (shaded blue) for each function along the slice $x_2 = 0.52$. While we would not expect the surrogates to perform well with only 5 observations, these figures still offer a useful visual of the GP's nonlinear regression and UQ capabilities. We will revisit the other features of Figures 1–2 in later sections.

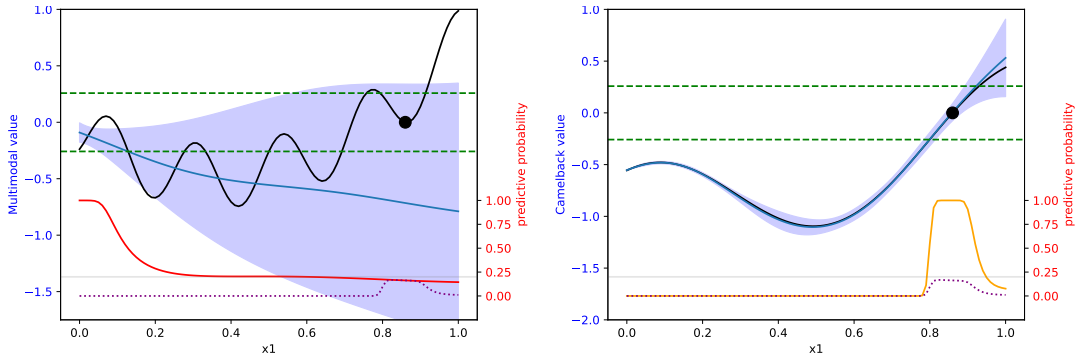
When response surfaces are nonstationary—meaning the covariance between two observations depends on more than just the Euclidean distance between locations—deep Gaussian processes (DGPs) offer superior performance. DGPs feature a "hierarchical cascading of Gaussians" (Dunlop et al., 2018), in which outputs of one GP feed as inputs to another. Inspired by spatial warpings (Sampson and Guttorp, 1992), and akin to neural networks, the latent layers of a DGP warp the input space into a plausibly stationary regime, but their multidimensional functional form poses a challenge to posterior inference. To circumvent the difficult task of posterior integration, many have embraced approximate variational inference (e.g., Damianou and Lawrence, 2013; Salimbeni and Deisenroth, 2017; Marmin and Filippone, 2022). Yet when data sizes are limited and UQ is essential, Bayesian approaches that use elliptical slice sampling (ESS; Murray et al., 2010) to infer the latent warpings are preferable (Sauer et al., 2023b; Ming et al., 2023; Sauer et al., 2023a). The DGP's posterior distribution is conditionally Gaussian given ESS samples of latent layers, which enables the calculation of posterior probabilities through careful application of the standard Gaussian CDF (Booth et al., 2025).

When modeling multiple responses, it may be beneficial to incorporate correlations among $f^{(r)}(\mathbf{x})$ for $r = 1, \ldots, R$. Multitask GPs (Bonilla et al., 2007), also called "multi output GPs" or "vector valued GPs," expand the GP's prior covariance to include all pairings across locations ($i = 1, \ldots, n$) and across functions ($r = 1, \ldots, R$) (Alvarez et al., 2012). The posterior distribution is still Gaussian, and posterior

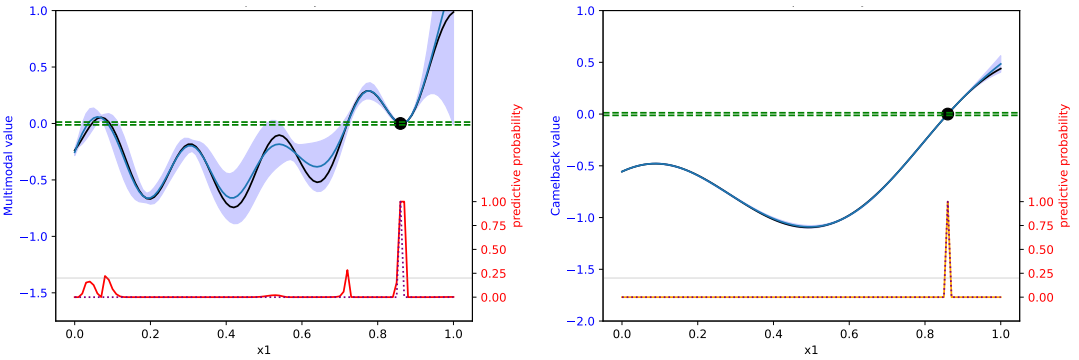
[1] Formulaic details of all synthetic functions and domains are provided in Supplementary Material.



(a) 5 LHS points + 0 acquired points from jCL



(b) 5 LHS points + 7 acquired points from jCL



(c) 5 LHS points + 20 acquired points from jCL

Figure 2: Slices at $x_2 = 0.52$ for the multimodal (left) and camelback (right) functions. Black line shows true surface; black circle highlights $\tilde{\mathbf{x}}$ where both functions return zero. Independent GP surrogates shown in blue starting with a 5-point LHS and proceeding with 0/7/20 jCL acquisitions across the two dimensions. Horizontal green dashed lines mark the target ($\tau_1 = \tau_2 = 0$) plus/minus the tolerance (t_n, Eq. 4). Red (multimodal) and orange (camelback) lines along the x-axis show the separate probabilities $\mathbb{P}(-t_n < f^{(r)}(\mathbf{x}) < t_n)$, on a relative scale. Purple dotted lines show the resulting joint probabilities. Joint probabilities that do not surpass $p^\star = 0.2$ (horizontal gray line) will trigger an exploration step.

probabilities can again be calculated with careful application of Gaussian CDFs. Extensions of multitask methods to deep Gaussian processes are still on the frontier; we are not aware of any publicly available

software implementations. Notably, our jCL active learning procedure is plug-and-play with any surrogate that can provide posterior predictive distributions. We will demonstrate applicability to GPs, DGPs, and multitask GPs (which we will broadly refer to as "GP-based surrogates") in Sections 4 and 5.

3 Joint Contour Location

We seek an objective acquisition procedure that targets the optimal design point $\tilde{\mathbf{x}}$, such as the red triangle indicated in Figure 1, with as few evaluations of the expensive functions as possible. Throughout, we consider evaluation of $f^{(r)}(\mathbf{x})$ for all $r = 1, \ldots, R$ as a single observation. Any time we evaluate a particular $\mathbf{x}$, we observe all responses at that same location. Our objective contrasts existing CL methodologies that target an entire contour. Figure 1 visualizes this disconnect; spending expensive acquisitions to learn the entirety of each contour (solid/dashed white) is not the most efficient or effective way to pin down the optimal design point (red triangle).

We propose two distinct acquisition strategies—one targeting exploitation of the optimal design point and one providing exploration of the response surfaces—with a decision rule to determine which to use for each acquisition. In this section, we will introduce these strategies separately before integrating them into one seamless AL procedure. At each acquisition step, we presume to have observed n data points and have trained GP-based surrogates, $f_n^{(r)}$ for $r = 1, \ldots, R$. Our objective is to determine which input, $\mathbf{x}_{n+1}$, to observe next.

3.1 Exploitation

A reasonable exploitive acquisition is any input which, with high probability given existing surrogate knowledge, could return τ_r for all $r = 1, \ldots, R$. [The ideal acquisition would of course be the exact $\tilde{\mathbf{x}}$.] Since we are operating with a continuous response variable, we must consider probabilities over intervals rather than probabilities of exact outcomes. Let t represent some tolerance value. For independent surrogates trained on n observations, the joint posterior probability of a particular input residing within tolerance for all functions is

$$J_n(\mathbf{x}, t) = \prod_{r=1}^{R} \mathbb{P}\left(\tau_r - t \leq f_n^{(r)}(\mathbf{x}) \leq \tau_r + t\right). \tag{3}$$

With GP-based surrogates, these probabilities involve Gaussian CDF computations. This form may be easily modified for multitask surrogates with correlations among responses. For simplicity, we use a constant tolerance across all functions after scaling each function to equivalent ranges, but our methodology could easily accommodate unique t_r.

This "joint probability" (for short) is a natural acquisition function for a sequential design targeting the intersection of multiple contours. Simply acquire

$$\mathbf{x}_{n+1} = \arg\max_{\mathbf{x} \in \mathcal{X}} \; J_n(\mathbf{x}, t).$$

Yet the choice of tolerance t can make-or-break this criterion. If t is too small, the $J_n(\mathbf{x}, t)$ criterion may be entirely flat (with the surrogates not anticipating any inputs will meet this stringent tolerance) or may be too peaky (with only a tiny portion of the input space offering a nonzero probability), which could thwart standard optimizations. If t is too large, we run the risk of one of our observed data points actually falling within tolerance across the board. If an observed $\mathbf{x}_i$ for $i \in \{1, \ldots, n\}$ satisfies $\tau_r - t \leq f^{(r)}(\mathbf{x}_i) \leq \tau_r + t$ for all $r = 1, \ldots, R$, then, given the deterministic nature of our computer experiments, we will have $J_n(\mathbf{x_i}, t) = 1$,

which is the highest possible joint probability. In that case, the maximum joint probability is guaranteed to reside at $\mathbf{x}_i$, but $\mathbf{x}_i$ is already in our training data and would be a waste of an acquisition.

We propose an adaptive refinement of the tolerance which: (1) gradually converges to zero to guide acquisitions closer and closer to the joint contour and (2) guarantees $J_n(\mathbf{x}_i, t)$ will be approximately zero for any observed $\mathbf{x}_i \in \mathbf{X}_n$ in order to discourage duplicate observations, which are wasteful in our deterministic setting. Specifically, we set

$$t_n = w \cdot \left[\min_{i=1,\ldots,n} \left(\max_{1 \le r \le R} |y_{ir} - \tau_r| \right) \right] \quad \text{where} \quad y_{ir} = f^{(r)}(\mathbf{x}_i) \quad \text{and} \quad 0 < w < 1. \tag{4}$$

Acquisitions use $J_n(\mathbf{x}, t_n)$, which is adjusted as acquisitions are made and n is incremented. Let's break this down from the inside out. First, we take each observed response value and find its distance from the contour. For a particular observation, we pull the largest of these distances across all functions. This is a measure of the "worst-case" result for a particular input. For example, even if $f^{(1)}(\mathbf{x}) = \tau_r$ exactly, if $f^{(2)}(\mathbf{x}) = \tau_r + 100$, then the quantity $\max_{1 \le r \le 2} |y_{ir} - \tau_r|$ will equal 100. Then, we pull the minimum of all these "worst-case" distances across all the observed data points. Think of this as the "best of the worst." For our "best" observed data point (the one that got the closest to having all response values equal τ_r), we grab the distance that was furthest off from all the functions within that data point. Then, finally, we shrink that distance by a factor w which must be strictly less than 1. This tolerance specification ensures any observed $\mathbf{x}_i \in \mathbf{X}_n$ will not be targeted since it will have near zero $J_n(\mathbf{x}_i, t_n)$ because $f_n^{(r)}(\mathbf{x})$ will equal $f^{(r)}(\mathbf{x})$ in our deterministic setting (as long as the surrogate properly preserves interpolation). Crucially, t_n will only shrink as n is incremented if a "better" point is acquired (meaning its $\max_{1 \le r \le R} |y_r - \tau_r|$ is lower than the previous low). While the joint probability surface is driven by surrogate predictions, the setting of t_n is independent of the surrogate. For continuous response surfaces, this iterative shrinking of the tolerance will greedily guide acquisitions toward the optimal $\tilde{\mathbf{x}}$.

The scaling parameter w is a user-adjustable tuning parameter that controls how aggressively the tolerance is tightened after each "better" observation is found. Values that are too large (i.e., $w \approx 1$) can slow progress. Values that are too small (i.e., $w < 0.5$) can cause the regions of non-zero joint probability to shrink too quickly, making the surface more difficult to optimize. We find $w = 0.9$ works well. We use this default in all of our exercises, with further comparisons reserved for Supplement C. We also acknowledge that variations of Eq. (4) which leverage different distance metrics might work equally well, although we favor the interpretability of the absolute value.

Returning to the illustrative example of Figure 2, the dashed green lines show $\pm t_5$, $\pm t_{12}$, and $\pm t_{25}$ chosen according to Eq. (4). As the design progresses, the tolerance shrinks, honing in on the contour. The independent probabilities, $\mathbb{P}\left(\tau_r - t_n \le f_n^{(r)}(\mathbf{x}) \le \tau_r + t_n\right)$ for $r \in \{1, 2\}$, are shown by the solid red/orange lines along the x-axis. They capture the surrogate's belief that the response will fall within the tolerance bounds for a given $\mathbf{x}$. The joint probability formed by the product of these is displayed in dotted purple. The next acquisition would be at the peak of this joint probability (albeit in two dimensions – the slice is just for illustration). Notice the joint probability focuses on regions where both probabilities are high; it is not led astray by inputs that have high probabilities for the multimodal function but not the camelback function, or vice versa. Larger tolerances are necessary early on when the surrogates do not have a lot of information regarding the location of $\tilde{\mathbf{x}}$. As closer observations are acquired, shrinking the tolerance facilitates effective exploitation as the joint probability peak converges around the true $\tilde{\mathbf{x}}$ (purple circle).

As another visual, Figure 3 shows $J_n(\mathbf{x}, t_n)$ in the full two-dimensional space. The left panel shows the joint probability after 4 acquisitions ($n = 9$ with 5 initial LHS points), resulting in the selection of the red triangle. The right panel shows the updated joint probability after the incorporation of the acquired point. Even though the surrogates are not accurately identifying the contours (the red predicted contours are not

great matches to the white true contours), the acquisition surface which incorporates their uncertainty is useful. Also notice that the strategic shrinking of the tolerance ensures that the joint probability will not be optimized at the previously observed location.

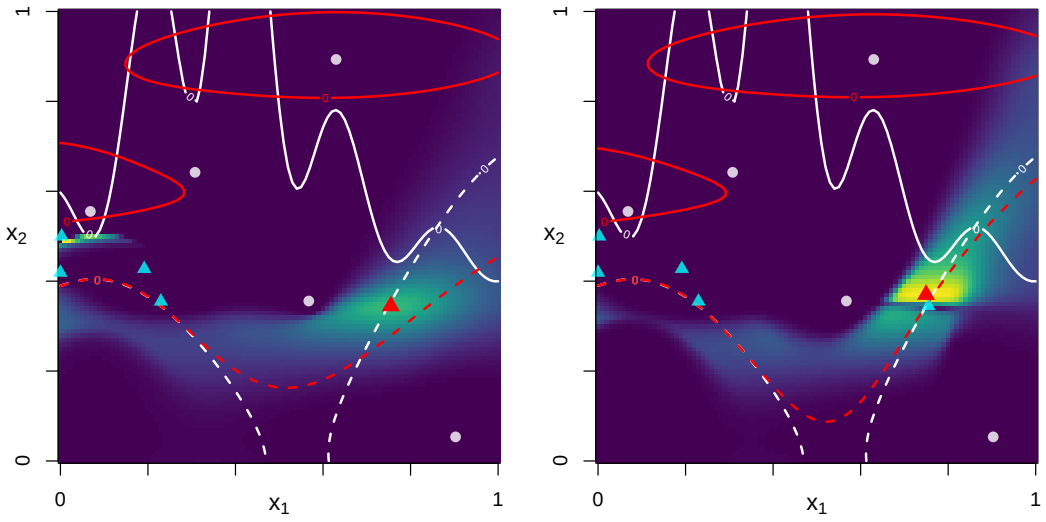


Figure 3: Two iterations of exploitation acquisitions for the multimodal and camelback functions. Heatmaps (yellow/high, purple/low) show $J_n(\mathbf{x}, t_n)$ after 4 jCL acquisitions (left) and 5 jCL acquisitions (right). White solid/dashed lines show the separate contours; red solid/dashed lines show the predicted contours. White circles indicate an initial LHS, and cyan triangles indicate exploitation acquisitions made by our jCL scheme thus far. The red triangle marks the next acquisition, where the joint probability is maximized.

3.2 Exploration

While the joint probability offers effective exploitation of the joint contour, sole reliance on it could be futile. There may be times throughout the design when the surrogates are not confident that any input would fall within tolerance for all functions, and the joint probability surface may be uninformative. This could occur due to low accuracy, high uncertainty, or both. It may also be a sign that no $\tilde{\mathbf{x}}$ exists (more on that momentarily). In this case, when we do not have confidence in our ability to *exploit*, we would rather *explore*.

To quantify this intuition, we set a threshold $p^\star \in (0, 1)$ and propose triggering exploratory acquisitions when $\max_{\mathbf{x}\in\mathcal{X}} J_n(\mathbf{x}, t_n) < p^\star$. The $p^\star$ parameter is another user-controlled tuning parameter indicating what probabilities would be considered large enough to indicate confidence in an exploitative acquisition. Larger $p^\star$ values are more conservative and will trigger more exploration and less exploitation. We find $p^\star = 0.2$ works well, and we use this default in all our exercises. Further comparisons are provided in Supplement C. To see this in action, the horizontal grey lines in Figure 2 mark $p^\star = 0.2$. In the center panels, the joint probability does not surpass this threshold, which we take as an indication that exploration at this stage would be beneficial.

A reasonable exploratory acquisition will increase accuracy and reduce uncertainty across the surrogates for all functions. We thus target locations where surrogate uncertainty is high for all $r = 1, \ldots, R$. This choice is inspired by previous works on sequential design for variance reduction, such as those leveraging

active learning MacKay, active learning Cohn, and integrated mean squared error acquisition functions (as reviewed in Gramacy, 2020, Chapter 6.2). Let $\sigma_n^{(r)}(\mathbf{x})$ represent the posterior standard deviation for the rth function at input $\mathbf{x}$, given n observed data points. [For a GP surrogate, this would take the form of σ in Eq. (2)]. We seek an acquisition that has high $\sigma_n^{(r)}(\mathbf{x})$ for all $r = 1, \ldots, R$. It is highly unlikely that all of these standard deviations would be optimized at the same $\mathbf{x}$, so we instead target their Pareto front. Let $c(\mathbf{x}) = \{\sigma_n^{(1)}(\mathbf{x}), \cdots, \sigma_n^{(R)}(\mathbf{x})\}$ denote our R-dimensional criterion of interest. An input $\mathbf{x}'$ is said to "dominate" $\mathbf{x}$ if $\sigma_n^{(r)}(\mathbf{x}') \geq \sigma_n^{(r)}(\mathbf{x})\ \forall\ r = 1, \ldots, R$. The Pareto front of $c(\mathbf{x})$ contains all $\mathbf{x} \in \mathcal{X}$ which are "non-dominated," meaning there are no other inputs that dominate them, as demonstrated in the right panel of Figure 4.

Although the Pareto front is technically a continuum, we prefer to work over a discrete set of candidates. To ensure candidates are placed in regions of high uncertainty, we use "triangulation candidates" (tricands; Gramacy et al., 2022). To explain, we offer a demonstration in Figure 4. The heatmaps in the left and center panels show $\sigma_n^{(r)}$ for the multimodal and camelback surfaces with $n = 12$ (continued from previous figures). We already saw in Figure 2(b) that the joint probability at this stage did not exceed $p^\star$, so an exploratory acquisition is warranted. The white circles in Figure 4 represent the observed inputs thus far. Starting with a Delaunay triangulation of the existing locations (solid white lines), tricands (red diamonds and triangles) are proposed at the interior of each triangle and extending from facets of the convex hull. By intentionally spreading candidates away from observed locations, tricands are likely to end up in regions of high uncertainty.

To choose an acquisition from the proposed tricands, we select a tricand from the Pareto front of $c(\mathbf{x})$ as defined above. The right panel of Figure 4 visualizes the $c(\mathbf{x})$ criterion across the proposed tricands, with candidates on the Pareto front as triangles. Notice how these locations have comparatively high uncertainties for both functions, as seen in the left/center panels. The magnitude of each $\sigma_n^{(r)}$ is only relevant within r, not across r, another distinct advantage of the Pareto front strategy. When there are multiple candidates on the Pareto front, we follow Booth et al. (2025) in selecting one at random.

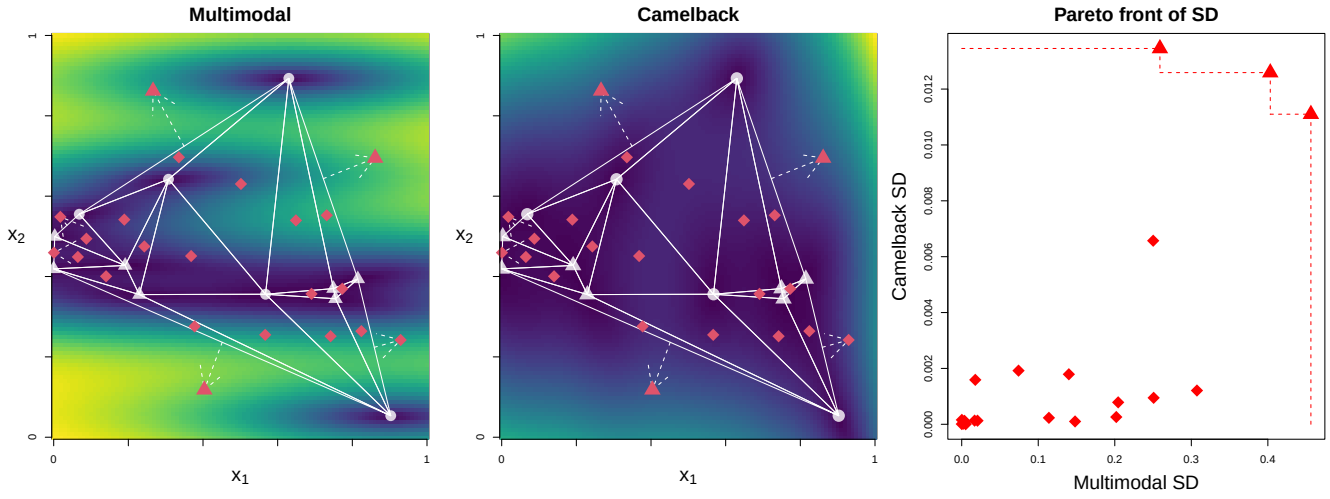


Figure 4: *Left/Center:* Heatmaps showing posterior predictive standard deviations (SD) for the multimodal and camelback functions after 12 observations (5 LHS + 7 jCL). White lines show Delaunay triangulation of observed locations. Red points show tricands. *Right:* SD's for each tricand. Across all panels, the candidates on the Pareto front are marked with triangles.

3.3 Putting it all together

Here, we summarize our comprehensive jCL procedure (we will provide specific implementation details in Section 4.1). We start by acquiring an initial space-filling design of size n and training GP-based surrogates for each function. Then, a single acquisition proceeds as follows. Set t_n following Eq. (4). Identify $\max_{\mathbf{x}\in\mathcal{X}} J_n(\mathbf{x}, t_n)$. If this maximum exceeds $p^\star$, then exploit by acquiring the $\mathbf{x}_{n+1}$ that yielded this maximum. If this maximum does not exceed $p^\star$, then explore by proposing tricands and acquiring the candidate on the Pareto front of $\{\sigma_n^{(1)}(\mathbf{x}), \cdots, \sigma_n^{(R)}(\mathbf{x})\}$. Observe $f^{(r)}(\mathbf{x}_{n+1})$ for each r, update the surrogates with this new data, and repeat until the budget is exhausted or a stopping rule is satisfied. In short, exploitation acquisitions are the default, and exploration is only triggered as a "fail-safe" if exploitation does not look promising (i.e. $\max_{\mathbf{x}\in\mathcal{X}} \; J_n(\mathbf{x}, t_n) < p^\star$).

Now we finally return to Figure 1, which shows a complete jCL sequential design. Cyan triangles indicate exploitative acquisitions, and cyan squares indicate explorative acquisitions. The first acquisitions were near the contours at low values of x_1; these observations helped the surrogates learn that the contours, although nearing each other, did not intersect in that region. Then there were several exploratory steps, all selecting locations far from others providing useful information about the general shape of the surfaces. Ultimately, jCL acquisitions were able to pinpoint the region of $\tilde{\mathbf{x}}$ (red triangle) and hone in on it effectively, thanks to the gradual shrinking of the tolerance as new acquisitions got closer and closer to the optimal design point.

3.4 Zero or multiple solutions

Although our focus is on identifying a single $\tilde{\mathbf{x}}$, it is prudent to discuss cases where zero or multiple solutions exist. Our procedure is designed to locate a single solution as efficiently as possible. If multiple solutions exist, jCL should find one solution but is not guaranteed to explore others. To demonstrate, we altered the camelback function shown earlier in Figure 1 so its contour intersects the multimodal function's contour in three places, as shown in the left panel of Figure 5. The jCL design (cyan squares/triangles) finds the rightmost intersection, but then terminates the design without exploring the other two. Full identification of all solutions will require further methodological development. We leave this as an interesting topic for future work, which we will discuss in Section 6.

Our jCL procedure is well-suited to handle situations where no optimal design point exists. In these cases, jCL will tend towards successive explorative acquisitions. If a certain number of exploration steps are triggered in a row because no promising directions for exploitation have been found, it is reasonable to halt the design. A sensible value for this number is problem dependent; it should take into account the severity of missing a solution as well as the surface complexity and input dimension (substantial exploration may be warranted for complex surfaces).

To demonstrate, we again altered the camelback function, this time so its contour no longer intersects with the multimodal function's contour, as shown in the right panel of Figure 5. Cyan squares/triangles again show explorative/exploitative acquisitions. For further context, Figure 6 shows the order in which these two classes of acquisitions were made. The procedure starts with some exploitative acquisitions where the contours near each other ($x_1 \approx 1$, $x_2 \approx 0.3$), but the surrogates use these acquisitions to learn that there is actually no intersection in that region. By the 10th iteration, every single acquisition is explorative. We continue for 50 acquisitions (much longer than necessary) to show how acquisitions continue to explore without finding any new regions for exploitation. The design could reasonably be halted after 10 successive explorative acquisitions, with confidence that no solution exists.

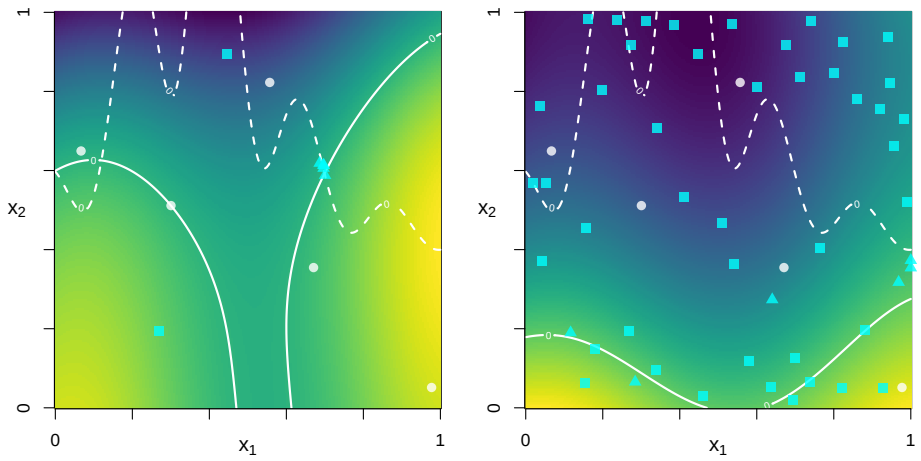


Figure 5: Modification of the 2d camelback function so multiple solutions (left) and no solutions (right) exist. White lines shows the separate contours of the multimodal and camelback functions. White circles indicate an initial LHS, and cyan squares (exploration) and triangles (exploitation) indicate acquisitions made by our jCL scheme. The sampling budget is increased to 50 for this demonstration.

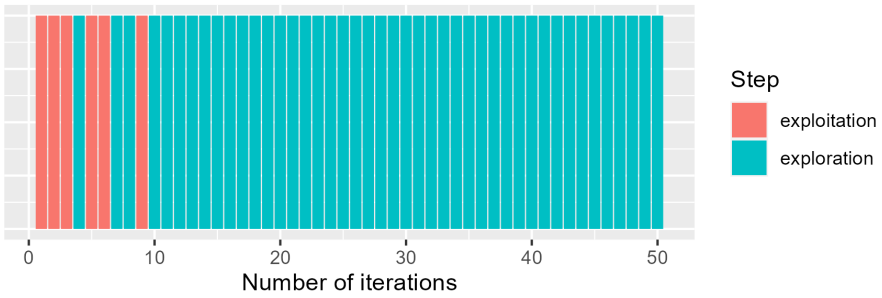


Figure 6: Order of acquisitions made for the jCL design shown in the right panel of Figure 5.

4 Benchmarking

In this section, we will validate jCL's performance against state-of-the-art alternatives on a variety of benchmark exercises. We measure performance using the Euclidean distance between the target $\{\tau_1, \ldots, \tau_r\}$ and the best observed point in y-space. Specifically, denote the "best squared Euclidean distance" after n observations as

$$D_n = \min_{i \in \{1,\ldots,n\}} \sum_{r=1}^{R} (y_{ir} - \tau_r)^2. \tag{5}$$

Lower D_n indicates better performance in identifying $\tilde{\mathbf{x}}$ as defined in Eq. (1). This performance metric, akin to simple regret in traditional Bayesian optimization, relies only on observed data and does not involve the surrogate. It is possible to leverage the surrogate to quantify uncertainty in the predicted optimal design point, but we prefer this "best observed point" metric as it provides a controlled comparison across all methods. With an eye towards our motivating application, we focus on lower dimensional problems with small training budgets, but we discuss extensions to larger exercises in Section 6. Reproducible code for

all benchmark exercises is available in our public git repository.[2]

4.1 Implementation Details

In this section, to demonstrate the applicability of our procedure, we will deploy both traditional GP and nonstationary DGP surrogates. We train independent surrogates for each function, knowing that the benchmark functions are indeed independent. For GPs, we use the `GPy` Python-package (GPy, 2025) with maximum likelihood estimation of kernel hyperparameters. For DGPs, we use the `deepgp` R-package (Booth, 2024) which employs ESS sampling of latent layers with Metropolis-Hastings sampling of kernel hyperparameters. For all surrogates, we use Matèrn-5/2 kernels and a fixed nugget of 1.0×10^{-6} to reflect the deterministic nature of each function. Otherwise, we use software defaults. We implement all exercises in Python, using the `RPy2` package (Gautier, 2024) to access the `deepgp` functions within Python.

To optimize the joint probability of Eq. (3), we employ a multi-start numerical optimization using the `minimize` function from the `SciPy` Python-package (Virtanen et al., 2020). Multi-start initializations are seeded with the best $10d$ out of $10,000d$ random points, the optimal joint probability point from the previous iteration, and an additional $10d$ random points. We also use a logarithmic trick to avoid instability in the joint probability calculation when the tolerance is small. Instead of optimizing $J_n(\mathbf{x}, t_n)$ directly, we optimize

$$\sum_{r=1}^{R} \left\{ \ln\left(\mathbb{P}\left(\tau_r - t \leq f_n^{(r)}(\mathbf{x}) \leq \tau_r + t \right) \right) \right\} = \sum_{r=1}^{R} \left\{ \ln\left(\underbrace{\mathbb{P}(f_n^{(r)}(\mathbf{x}) \leq \tau_r + t)}_{=:a} - \underbrace{\mathbb{P}(f_n^{(r)}(\mathbf{x}) \leq \tau_r - t)}_{=:b} \right) \right\}$$
$$= \sum_{r=1}^{R} \left\{ \ln(a) + \ln\left(1 - \exp\left(\ln(b) - \ln(a) \right) \right) \right\}.$$

We use the `SciPy` Python-package (Virtanen et al., 2020) to calculate Delaunay triangulations and propose tricands following Gramacy et al. (2022).

In the following exercises, we halt the sequential design when either the total budget of evaluations has been spent or an acceptable $D_n < \epsilon$ has been reached. Throughout, we use $\epsilon = 0.001$ after appropriately scaling the responses (scaling details are provided in Supplement A).

4.2 Competitors

We consider six alternative designs (within each exercise, surrogate choices are kept consistent, the only variations are the methods for selecting training data). First, we benchmark our jCL design against a space-filling Latin hypercube sample of equivalent size. Although we will report LHS performance as the sample size is incremented, the LHS design is not sequential and does not use the surrogates to select design locations. Next, we compare to two state-of-the-art contour location schemes: that of Cole et al. (2023) and Booth et al. (2025). Cole et al. acquire points based on a local optimization of the surrogate's classification entropy, which can be heavy on exploitation. Booth et al. encourage more exploration by acquiring triangulation candidates on the Pareto front of entropy and uncertainty. Both of these methods have been shown to excel at contour location, but are designed for a single function. To suit them to our setting, we simply alternate between CL acquisitions for each function. We refer to these methods as "alternating entropy" and "alternating Pareto," respectively.

[2] `https://bitbucket.org/boothlab/jcl`

Next, we compare against two common optimization algorithms: Bayesian optimization ("BO") and zero-finding ("Zero"). To leverage these methods, we transform the problem of Eq. (1) into the following optimization problem:

$$\tilde{\mathbf{x}} = \underset{\mathbf{x}\in\mathcal{X}}{\operatorname{argmin}}\, h(\mathbf{x}) \quad \text{where} \quad h(\mathbf{x}) = \sum_{r=1}^{R} \left(f^{(r)}(\mathbf{x}) - \tau_r \right)^2.$$

The BO procedure trains a single GP/DGP surrogate on observed $\{\mathbf{x}_i, h(\mathbf{x}_i)\}$ for $i = 1, \ldots, n$ then uses the expected improvement criterion (Jones et al., 1998) to select acquisitions in order to minimize $h(\mathbf{x})$. We use the "L-BFGS-B" algorithm (Byrd et al., 1995) with $10d$ randomly seeded multi-starts to optimize the EI surface for each acquisition. The zero-finding procedure uses L-BFGS-B directly on the black-box $h(\mathbf{x})$ function without any surrogate. When assessing performance, we carefully track the number of black-box evaluations used by the zero-finding method in order to match it appropriately to the sequential surrogate-informed methods.

Lastly, we implement a joint acquisition function inspired by the "targeted adaptive design" procedure of Graziani and Ngom (2024), but without the extra machinery they built-in to accommodate noisy simulations and wider target ranges. This method, which we label "PDF," selects acquisitions by optimizing the surrogates' joint probability density function centered at the target $(\tau_1, \ldots, \tau_r)$. For independent GP surrogates with posterior means $\mu_n^{(r)}(\mathbf{x})$ and posterior standard deviations $\sigma_n^{(r)}(\mathbf{x})$, acquisitions are chosen following

$$\mathbf{x}_{n+1} = \underset{\mathbf{x}\in\mathcal{X}}{\operatorname{argmin}}\, g(\mathbf{x}) \quad \text{where} \quad g(\mathbf{x}) = \prod_{r=1}^{R} \phi\left(\frac{\tau_r - \mu_n^{(r)}(\mathbf{x})}{\sigma_n^{(r)}(\mathbf{x})} \right),$$

and ϕ represents the standard Gaussian PDF. This acquisition function is similar to our joint probability metric, but it omits the shrinking tolerance t_n and loses the interpretability of the probability, which is key to informing our exploratory acquisitions. To maintain fair comparisons, we optimize the PDF acquisition function with the same optimization procedures used for the aforementioned methods.

4.3 Results

We consider three test cases. In each setting, we prescale inputs to the unit cube, i.e., $\mathcal{X} = [0, 1]^d$, and prescale responses to have unit variance. We use $\tau_r = 0$ for all r, after shifting the functions so there exists a single $\tilde{\mathbf{x}}$ (details provided in Supplement A). First, we continue the 2d multimodal and camelback exercise of Figures 1–4 (with $d = R = 2$). We use GP surrogates with an initial LHS of size $n_0 = 5$ and a total budget of $n = 25$. Second, we use two variations of the 2d "Gramacy" function (Gramacy and Lee, 2009), which is characterized by large flat regions with a single "hill" and "valley." A visual of these surfaces and one of our jCL designs is provided in Supplement B. To accommodate the nonstationarity of the surfaces, we use DGP surrogates, again with $d = R = 2$, $n_0 = 5$, and $n = 25$. Finally, we expand to $d = R = 3$ with an adaptation of the multimodal function, the Ishigami function (Ishigami and Homma, 1990), and a third "trig" function consisting of several sine and cosine functions. These surfaces are relatively stationary, so we return to GP surrogates. For this larger dimension, we start with an LHS of size $n_0 = 10$ and acquire up to $n = 40$. We repeat each exercise for 50 Monte Carlo repetitions with re-randomized starting designs.

Performance in D_n is shown in Figure 7. Our jCL design (solid teal) consistently outperforms the competitors. In all three examples, jCL has a much steeper decline in D_n. For the 2d multimodal and camelback functions, jCL achieves better median performance after 5 acquisitions than the other methods could achieve after 20. For the other examples, after about 10 acquisitions jCL achieves median performance that is lower than the "almost best" performance of the other methods. Our jCL design is the only one to

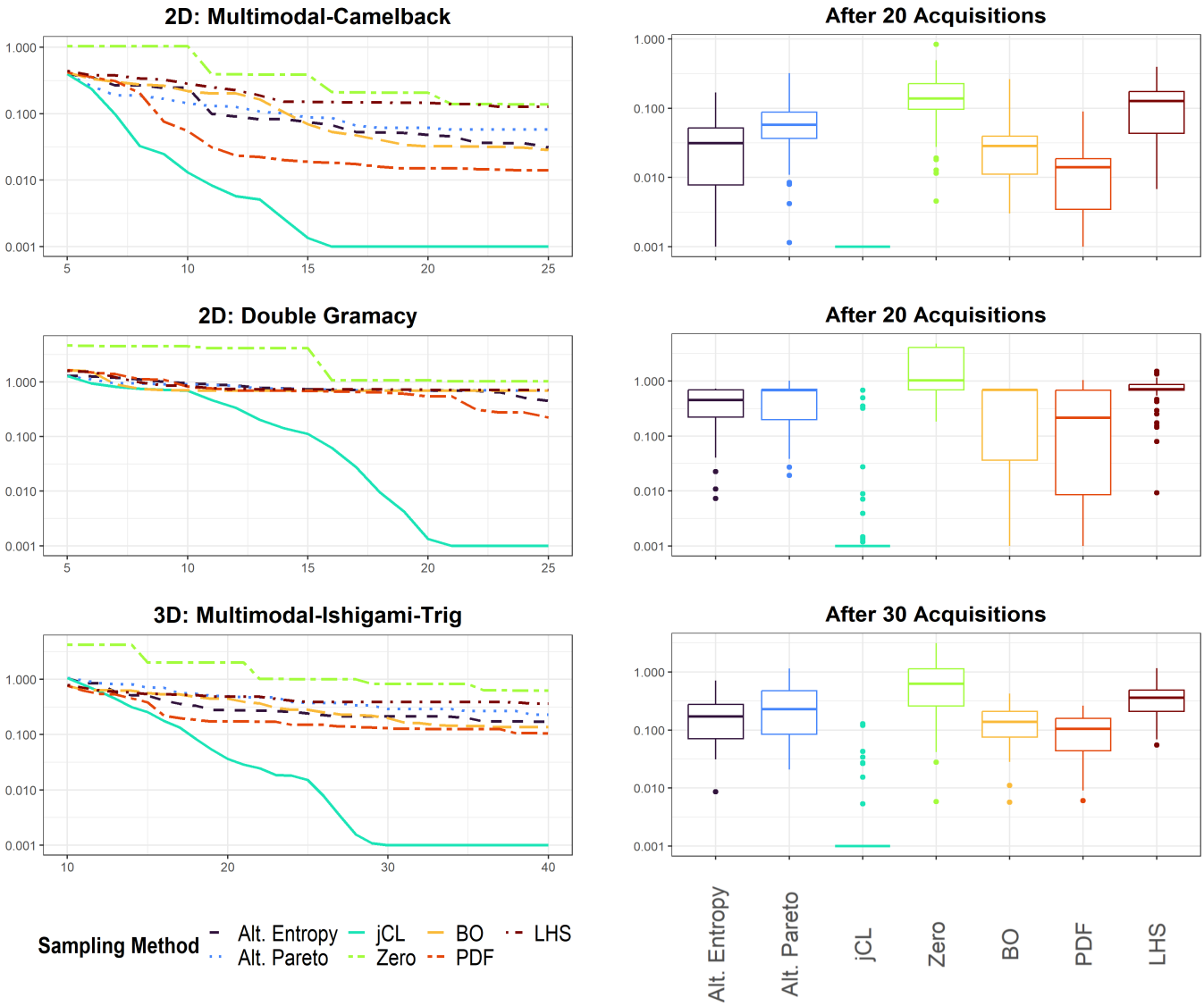


Figure 7: Performance in D_n (Eq. 5) on the log scale. Line plots show median performance as n is incremented. Box plots show the spread at the end of sequential sampling. Top and bottom panels use GP surrogates; center panel uses DGP surrogates.

consistently reach the specified ϵ, and it often does so before spending the maximum allowed budget. The alternating contour location methods struggle as they waste observations learning other non-intersecting regions of the contours. Of the optimization methods, BO which makes careful use of a surrogate model outperforms the zero-finding optimization algorithm, but it still pales in comparison to jCL. We suspect this is largely due to the complexity of the $h(\mathbf{x})$ surface compared to the separate $f^{(r)}(\mathbf{x})$ surfaces. The $h(\mathbf{x})$ is undoubtedly less smooth, more complex, and more challenging to learn than the separate functions. The PDF method is second-best, but it still struggles compared to jCL. Although the PDF acquisition function involves surrogate uncertainty, it does not have a strong mechanism for exploration. Its efforts to exploit can also be less aggressive than those of the joint probability thanks to the shrinking tolerance.

5 High-Speed Army Reference Vehicle

Here we consider our motivating application: CFD simulations of the rotational torques acting on the high-speed army reference vehicle across various free-stream conditions. We use NASA's CBAERO (which is preferred for its computation speed) in this study to generate numerical approximations of the forces and moments on the vehicle (Kinney, 2007). The left panel of Figure 8 shows the HARV geometry. The deflection of the fins are the key controllable inputs. We seek the optimal deflection of the fins ($\tilde{\mathbf{x}}$) that will result in stable flight for given flight conditions (Mach, angle of attack, etc.), with as few CBAERO evaluations as possible. Here, we focus on the configuration of the two side fins, targeting zero pitch moment and zero roll moment (we fix the top and bottom fins and the sideslip angle to focus on the longitudinal degrees of freedom).

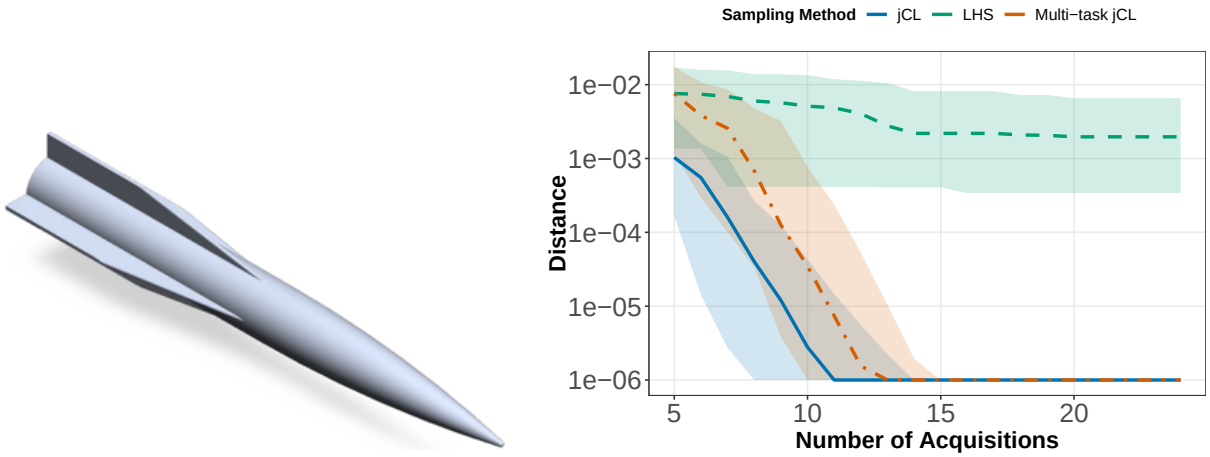


Figure 8: *Left:* HARV geometry featuring four deflectable fins. *Right:* Performance in D_n (Eq. 5) across acquisitions.

Since there is reason to suspect correlation between pitch and roll, we deploy our jCL procedure with both independent GP surrogates and a multitask GP surrogate (implemented in `botorch`; Balandat et al., 2020). We use an initial design of size $n_0 = 5$, a maximum budget of $n = 25$, and a stopping criterion of $\epsilon = 1 \times 10^{-6}$. We compared performance in D_n, omitting the poorer performers from Figure 7 but retaining the LHS as a baseline. Performance across 50 repetitions is shown in the right panel of Figure 8. Our jCL designs were able to consistently locate the optimal fin configuration with fewer than 15 observations. Curiously, the multitask GP slightly underperforms the independent GPs, but still quickly reaches the target accuracy. The space-filling designs made little-to-no progress. Finding the optimal wing configuration with only 10-15 observations enables us to repeat this procedure across a variety of flight conditions and top/bottom wing configurations in order to build out a trimmed aerodynamic database.

6 Conclusion

We have proposed a novel joint contour location (jCL) sequential design to identify the optimal design point from multiple computer experiments (Eq. 1). Our method uses the joint probability of the multiple responses being within a specified, converging tolerance in order to exploit. It uses triangulation candidates on the Pareto front of each surrogate's posterior standard deviation as a "failsafe" in order to explore when exploitation is not advised. Through careful, strategic design, jCL is able to outperform standard CL and

optimization-based designs with far fewer observations of the black-box computer experiments. Our jCL is also versatile; it can be implemented using any surrogate that provides posterior predictive distributions.

In this work we targeted lower dimensional settings ($d \leq 3$) with small evaluation budgets ($n \leq 40$). While our jCL procedure is applicable to larger scenarios, some modifications could be beneficial in higher dimensions. The triangulation candidates used in our exploration step may become cumbersome and ineffective for large d. Adaptations such as Voronoi candidates (Wycoff et al., 2024) could be advantageous. Furthermore, the Pareto front could become more diffuse and less informative as dimension grows. Modified strategies for exploratory acquisitions that still balance uncertainty across all functions may be warranted. Furthermore, we assumed that each $f^{(r)}$ would be evaluated together for the same input $\mathbf{x}$, which we treated as a single evaluation of the black-box function. If each $f^{(r)}$ is evaluated separately, then additional methodology to select which black-box function to query at each acquisition would be beneficial.

If multiple solutions exist (either at a discrete number of locations or on a continuum), our jCL procedure is designed to locate a single optimal design point efficiently. It will not ensure exploration of other solutions. We suspect strategic thresholds on the joint probability and adjustments to the tolerance refinement would facilitate identification of multiple solutions, but this would require careful investigation. For the HARV geometry, methods to identify multiple optima could facilitate application of jCL over the full parameter space, rather than for fixed flight conditions.

Acknowledgments

This work was performed under the auspices of the U.S. Department of Energy by Lawrence Livermore National Laboratory under Contract DE-AC52-07NA27344, LLNL-JRNL-2013503-DRAFT. The authors thank Brad Perfect for his invaluable suggestions and feedback.

Data Availability

Reproducible code including data generation for all synthetic exercises is provided in a public git repository at `https://bitbucket.org/boothlab/jcl`.

References

Alvarez, M. A., Rosasco, L., and Lawrence, N. D. (2012). "Kernels for vector-valued functions: A review." *Foundations and Trends® in Machine Learning*, 4, 3, 195–266.

Balandat, M., Karrer, B., Jiang, D., Daulton, S., Letham, B., Wilson, A. G., and Bakshy, E. (2020). "BoTorch: A framework for efficient Monte-Carlo Bayesian optimization." *Advances in neural information processing systems*, 33, 21524–21538.

Bect, J., Ginsbourger, D., Li, L., Picheny, V., and Vazquez, E. (2012). "Sequential design of computer experiments for the estimation of a probability of failure." *Statistics and Computing*, 22, 3, 773–793.

Belakaria, S., Letham, B., Doppa, J. R., Engelhardt, B., Ermon, S., and Bakshy, E. (2024). "Active learning for derivative-based global sensitivity analysis with Gaussian processes." *Advances in Neural Information Processing Systems*, 37, 53887–53917.

Bichon, B. J., Eldred, M. S., Swiler, L. P., Mahadevan, S., and McFarland, J. M. (2008). "Efficient global reliability analysis for nonlinear implicit performance functions." *AIAA journal*, 46, 10, 2459–2468.

Binois, M., Huang, J., Gramacy, R. B., and Ludkovski, M. (2019). "Replication or exploration? Sequential design for stochastic simulation experiments." *Technometrics*, 61, 1, 7–23.

Bonilla, E. V., Chai, K., and Williams, C. (2007). "Multi-task Gaussian process prediction." *Advances in neural information processing systems*, 20.

Booth, A. S. (2024). *deepgp: Bayesian deep Gaussian processes using MCMC*. R package version 1.1.3.

Booth, A. S., Cooper, A., and Gramacy, R. B. (2024). "Nonstationary Gaussian Process Surrogates." *arXiv preprint arXiv:2305.19242*.

Booth, A. S., Renganathan, S. A., and Gramacy, R. B. (2025). "Contour Location for Reliability in Airfoil Simulation Experiments Using Deep Gaussian Processes." *The Annals of Applied Statistics*, 19, 1, 191–211.

Byrd, R. H., Lu, P., Nocedal, J., and Zhu, C. (1995). "A Limited Memory Algorithm for Bound Constrained Optimization." *SIAM Journal on Scientific Computing*, 16, 5, 1190–1208.

Chevalier, C., Bect, J., Ginsbourger, D., Vazquez, E., Picheny, V., and Richet, Y. (2014). "Fast parallel kriging-based stepwise uncertainty reduction with application to the identification of an excursion set." *Technometrics*, 56, 4, 455–465.

Cohn, D. A., Ghahramani, Z., and Jordan, M. I. (1996). "Active learning with statistical models." *Journal of artificial intelligence research*, 4, 129–145.

Cole, D. A., Gramacy, R. B., Warner, J. E., Bomarito, G. F., Leser, P. E., and and, W. P. L. (2023). "Entropy-based adaptive design for contour finding and estimating reliability." *Journal of Quality Technology*, 55, 1, 43–60.

Damianou, A. and Lawrence, N. D. (2013). "Deep Gaussian Processes." In *Proceedings of the Sixteenth International Conference on Artificial Intelligence and Statistics*, eds. C. M. Carvalho and P. Ravikumar, vol. 31 of *Proceedings of Machine Learning Research*, 207–215. Scottsdale, Arizona, USA: PMLR.

Dunlop, M. M., Girolami, M. A., Stuart, A. M., and Teckentrup, A. L. (2018). "How Deep Are Deep Gaussian Processes?" *Journal of Machine Learning Research*, 19, 1–46.

Economon, T. D., Palacios, F., Copeland, S. R., Lukaczyk, T. W., and Alonso, J. J. (2016). "SU2: An open-source suite for multiphysics simulation and design." *Aiaa Journal*, 54, 3, 828–846.

Gautier, L. (2024). "RPy2: Python interface to the R language." `https://github.com/rpy2/rpy2/releases/tag/RELEASE_3_5_17`.

GPy (2025). "GPy: A Gaussian process framework in python." `http://github.com/SheffieldML/GPy`.

Gramacy, R. B. (2020). *Surrogates: Gaussian Process Modeling, Design and Optimization for the Applied Sciences*. Boca Raton, Florida: Chapman Hall/CRC. `http://bobby.gramacy.com/surrogates/`.

Gramacy, R. B. and Lee, H. K. (2009). "Adaptive design and analysis of supercomputer experiments." *Technometrics*, 51, 2, 130–145.

Gramacy, R. B., Sauer, A., and Wycoff, N. (2022). "Triangulation candidates for Bayesian optimization." *Advances in Neural Information Processing Systems*, 35, 35933–35945.

Graziani, C. and Ngom, M. (2024). "Targeted Adaptive Design." *SIAM/ASA Journal on Uncertainty Quantification*, 12, 4, 1273–1314.

Ishigami, T. and Homma, T. (1990). "An importance quantification technique in uncertainty analysis for computer models." In *[1990] Proceedings. First international symposium on uncertainty modeling and analysis*, 398–403. IEEE.

Jones, D. R., Schonlau, M., and Welch, W. J. (1998). "Efficient global optimization of expensive black-box functions." *Journal of Global optimization*, 13, 4, 455–492.

Kinney, D. (2007). "Aerothermal anchoring of CBAERO using high fidelity CFD." In *45th AIAA Aerospace Sciences Meeting and Exhibit*, 608.

Koermer, S., Loda, J., Noble, A., and Gramacy, R. B. (2024). "Augmenting a simulation campaign for hybrid computer model and field data experiments." *Technometrics*, 66, 4, 638–650.

Marmin, S. and Filippone, M. (2022). "Deep Gaussian processes for calibration of computer models (with discussion)." *Bayesian Analysis*, 17, 4, 1301–1350.

Marques, A. N., Lam, R. R., and Willcox, K. E. (2018). "Contour location via entropy reduction leveraging multiple information sources." *Advances in Neural Information Processing Systems*, 5217–5227.

McKay, M. D., Beckman, R. J., and Conover, W. J. (2000). "A comparison of three methods for selecting values of input variables in the analysis of output from a computer code." *Technometrics*, 42, 1, 55–61.

Ming, D., Williamson, D., and Guillas, S. (2023). "Deep Gaussian process emulation using stochastic imputation." *Technometrics*, 65, 2, 150–161.

Molga, M. and Smutnicki, C. (2005). "Test functions for optimization needs." *Test functions for optimization needs*, 101, 48, 32.

Murray, I., Adams, R. P., and MacKay, D. J. (2010). "Elliptical Slice Sampling." In *The Proceedings of the 13th International Conference on Artificial Intelligence and Statistics*, vol. 9 of *JMLR: W&CP*, 541–548.

Pamadi, B., Covell, P., Tartabini, P., and Murphy, K. (2004). "Aerodynamic characteristics and glide-back performance of langley glide-back booster." In *22nd Applied Aerodynamics Conference and Exhibit*, 5382.

Picheny, V., Ginsbourger, D., Roustant, O., Haftka, R. T., and Kim, N.-H. (2010). "Adaptive Designs of Experiments for Accurate Approximation of a Target Region." *Journal of Mechanical Design*, 132, 7, 071008.

Pourmohamad, T. and Lee, H. K. (2021). *Bayesian optimization with application to computer experiments*. Springer.

Quinlan, K. R., Movva, J., and Perfect, B. (2024). "Non-uniform active learning for Gaussian process models with applications to trajectory informed aerodynamic databases." *Statistical Analysis and Data Mining*, 17.

Rajaram, D., Puranik, T. G., Renganathan, A., Sung, W. J., Pinon-Fischer, O. J., Mavris, D. N., and Ramamurthy, A. (2020). "Deep Gaussian process enabled surrogate models for aerodynamic flows." In *AIAA scitech 2020 forum*, 1640.

Ranjan, P., Bingham, D., and Michailidis, G. (2008). "Sequential experiment design for contour estimation from complex computer codes." *Technometrics*, 50, 4, 527–541.

Rasmussen, C. E. and Williams, C. K. (2006). *Gaussian processes for machine learning*. MIT press Cambridge, MA.

Salimbeni, H. and Deisenroth, M. (2017). "Doubly stochastic variational inference for deep Gaussian processes." *Advances in neural information processing systems*, 30.

Sampson, P. D. and Guttorp, P. (1992). "Nonparametric Estimation of Nonstationary Spatial Covariance Structure." *Journal of the American Statistical Association*, 87, 417, 108–119.

Santner, T. J., Williams, B. J., Notz, W. I., and Williams, B. J. (2003). *The design and analysis of computer experiments*, vol. 1. Springer.

Sauer, A., Cooper, A., and Gramacy, R. B. (2023a). "Vecchia-approximated deep Gaussian processes for computer experiments." *Journal of Computational and Graphical Statistics*, 32, 3, 824–837.

Sauer, A., Gramacy, R. B., and Higdon, D. (2023b). "Active Learning for Deep Gaussian Process Surrogates." *Technometrics*, 65, 4–18.

Sauer, A. E. (2023). "Deep Gaussian process surrogates for computer experiments." Ph.D. thesis, Virginia Tech.

Song, D. and Joseph, V. R. (2026). "Efficient active learning strategies for computer experiments." *Technometrics*, 68, 1, 65–78.

Stein, M. L. (1999). *Interpolation of spatial data*. Springer-Verlag.

Sürer, Ö. (2025). "Simulation experiment design for calibration via active learning." *Journal of Quality Technology*, 57, 1, 16–34.

Vasile, J. D., Fresconi, F., DeSpirito, J., Duca, M., Recchia, T., Grantham, B., Bowersox, R. D., and White, E. B. (2022). "High-speed army reference vehicle." Tech. rep., ARLTN1128.

Vassberg, J., Dehaan, M., Rivers, M., and Wahls, R. (2008). "Development of a common research model for applied CFD validation studies." In *26th AIAA applied aerodynamics conference*, 6919.

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., et al. (2020). "SciPy 1.0: fundamental algorithms for scientific computing in Python." *Nature methods*, 17, 3, 261–272.

Wycoff, N., Binois, M., and Wild, S. M. (2021). "Sequential learning of active subspaces." *Journal of Computational and Graphical Statistics*, 30, 4, 1224–1237.

Wycoff, N., Smith, J. W., Booth, A. S., and Gramacy, R. B. (2024). "Voronoi candidates for bayesian optimization." *arXiv preprint arXiv:2402.04922*.

Yazdi, F., Bingham, D., and Williamson, D. (2024). "Deep Gaussian Process Emulation and Uncertainty Quantification for Large Computer Experiments." *arXiv preprint arXiv:2411.14690*.

SUPPLEMENTARY MATERIAL

A Synthetic Functions

In this section we provide details for the three test functions used in Section 4. We have adjusted and scaled each function to ensure that a single $\tilde{\mathbf{x}}$ exists at $\tau_r = 0$ for all $r \in \{1, \ldots, R\}$.

2D: Multimodal-Camelback

The "multimodal" function (Bichon et al., 2008) is defined as

$$y_1 = \frac{\frac{x_2-1}{20}\left(x_1^2+4\right) - \sin\left(\frac{5x_1}{2}\right) - 2}{3.556} - 0.00678656 \quad \text{for} \quad x_1 \in [-4, 7], \;\; x_2 \in [-3, 8].$$

The "camelback" function (Molga and Smutnicki, 2005) is defined as

$$y_2 = \frac{u^2\left(4 - 2.1u^2 + \frac{u^4}{3}\right) + 2uv + \left(\frac{26}{9}v^2\left(-4 + \frac{16v^2}{9}\right)\right) - 0.1}{2.242} - 0.00719266 \quad \text{for} \quad x_1 \in [-1, 1], \;\; x_2 \in [0, 1],$$

where $u = 1.2x_1 - 0.1$ and $v = 0.9x_2$.

2D: Double Gramacy

The "double Gramacy" functions (Gramacy and Lee, 2009) are defined as

$$y_1 = 9.27\left[x_1 \cdot \exp\left(\left(-x_1 - 0.5\right)^2 - \left(x_2 + 0.5\right)^2\right) - 0.1\right] + 0.09830494$$
$$y_2 = \frac{\left(x_2 + 0.5\right)\exp\left(-\left(\frac{x_2}{4}\right)^2 - x_1^2\right) - 1}{0.4975} - 0.01300846$$

For both functions, the domain is $x_i \in [-2, 6]$ for $i \in \{1, 2\}$.

3D: Multimodal-Ishigami-Trig

The 3-dimensional "multimodal" function is defined as

$$y_1 = \frac{\frac{x_2-1}{20}\left(x_1^2+4\right) - \sin\left(\frac{5x_1}{2}\right) - 2 + x_3 - 0.00052352}{3.588} \quad \text{for} \quad x_1 \in [-4, 7], \;\; x_2 \in [-3, 8], \;\; x_3 \in [0, 1].$$

The "Ishigami" function (Ishigami and Homma, 1990) is defined as

$$y_2 = \frac{\sin\left(x_1\right) + 7\sin\left(x_2\right)^2 + 0.1x_3^4\sin\left(x_1\right) + 2.79921514}{3.72} \quad \text{for} \quad x_i \in [-\pi, \pi] \; \forall \; i \in \{1, \ldots, 3\}.$$

The "Trig" function is defined as

$$y_3 = \frac{\sin(x_1) + \cos(x_1) + x_2^2 + \sqrt{x_3} + \sin(x_3) - 3.05174287}{0.582} \quad \text{for} \quad x_i \in [0,1] \; \forall \; i \in \{1,\ldots,3\}.$$

B Double Gramacy Design

Figure 9 shows the two surfaces of the "double Gramacy" example, with contours at $\tau_1 = \tau_2 = 0$ in solid/dashed white. Cyan points show a jCL design with a DGP surrogate. The combination of exploitation steps (triangles) and exploration steps (squares) effectively locates the optimal design point (red triangle).

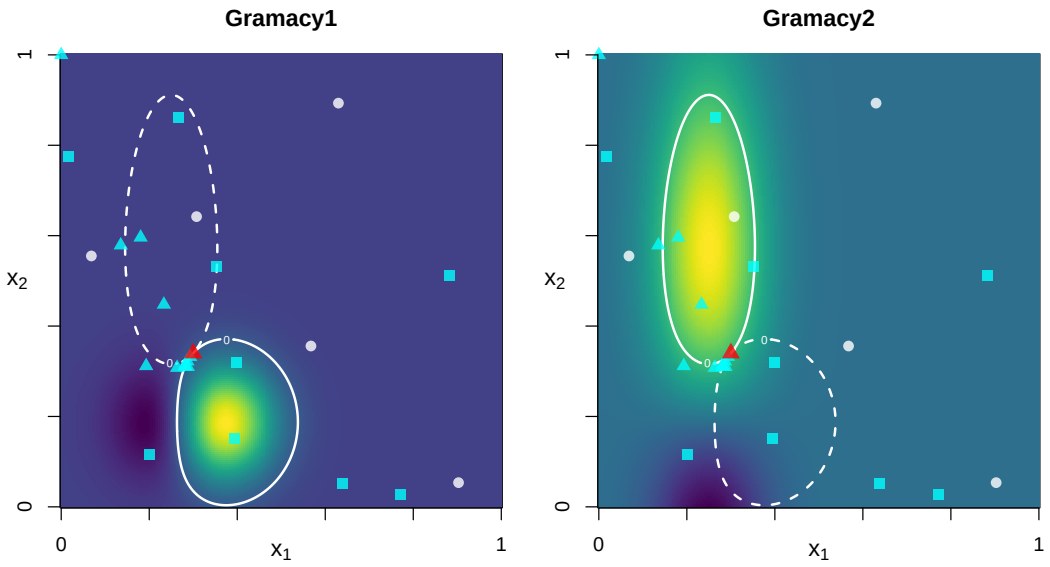


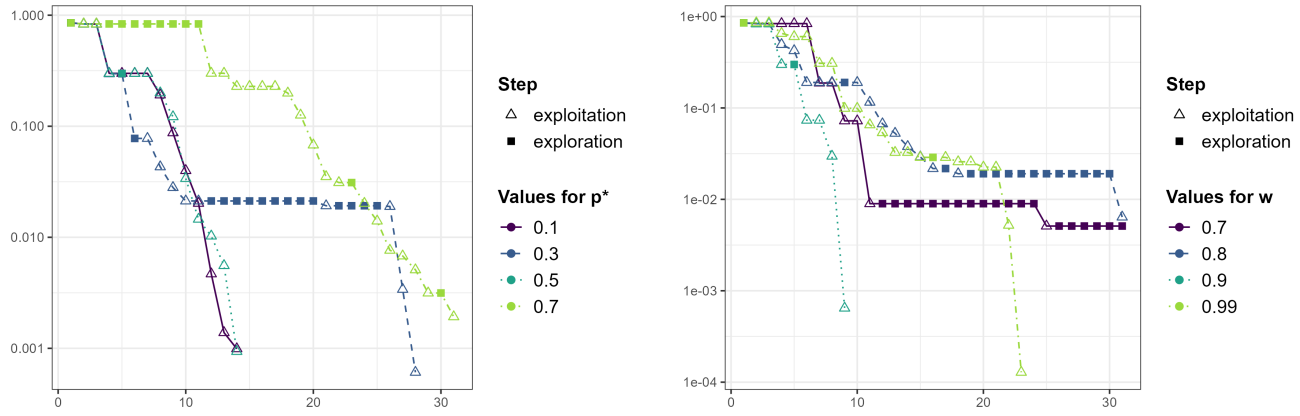
Figure 9: Heatmaps (yellow/high, purple/low) of the 2d Gramacy functions (derived from Gramacy and Lee, 2009). White lines shows the separate contours $f^{(r)}(\mathbf{x}) = 0$, and red triangle marks the optimal design point where both contours are zero. White circles indicate an initial LHS, and cyan squares (exploration) and triangles (exploitation) indicate acquisitions made by our jCL scheme.

C Examining tuning parameters

To examine the effect of $p^\star$ and w on our jCL procedure, we revisit the 3D Multimodal-Ishigami-Trig example of Section 4, this time varying the values of $p^\star$ and w while keeping the starting design constant. The left panel of Figure 10 shows performance in D_n for $p^\star \in \{0.1, 0.3, 0.5, 0.7\}$. Larger $p^\star$ values encourage more exploration. This is apparent from the design with $p^\star = 0.7$, which starts with many exploratory steps without making any progress. Eventually, the procedure will find an avenue for exploitation, but it may have wasted acquisitions in the process. There is variability in performance across lower $p^\star$ values caused by the unique acquisitions made by each design (with $p^\star = 0.1$ and $p^\star = 0.5$ performing similarly). Ultimately, we landed on a default of $p^\star = 0.2$, which will avoid excessive exploration without discouraging

it altogether, but we believe the interpretable and user-adjustable nature of this tuning parameter is a strength.

Next, we examine how the value of w affects jCL's performance. The tuning parameter w controls how much the tolerance t_n shrinks as each "better" design point is observed. A smaller value of w shrinks the tolerance more aggressively, and a larger value shrinks the tolerance more gradually. The only restriction is that w must be strictly less than 1. The right panel of Figure 10 shows results on the same 3D example with a variety of w values. Small values which are attempting to be more aggressive actually have the opposite effect; they shrink the tolerance too much, reducing the maximum joint probability and resulting in over-exploration. We caution against setting w too low and recommend $w = 0.9$ as a default.



(a) Varying values of p^* (using $w = 0.9$)

(b) Varying values of w (using $p^\star = 0.2$)

Figure 10: Performance in D_n for jCL designs of the 3D Multimodal-Ishigami-Trig example with varying tuning parameter values. All designs start with the same initial LHS. Squares indicate exploration steps, and triangles indicate exploitation steps.